To start working on this notebook, we will need to save our own copy of it. We can do this by clicking File > Save a Copy in Drive. We will then be able to make edits to our own copy of this notebook.

Python Data Cleaning: Basics II

1.0 Importing our Libraries

In [1]:
```
# Importing the Pandas Library
import pandas as pd

# Importing the Numpy Library
import numpy as np
```

1.1 Validity

In [2]:
```
# Example 1: Irrelevant Data
# Irrelevant data are those that are not actually needed, and don't fit under
# the context of the problem we're trying to solve.
# For example, we will drop the Team and Weight columns from the dataset below
# since we will not need them in our analysis
#

# Making the dataframe from csv file
data_df = pd.read_csv("http://bit.ly/MSDS-NBADataset", index_col ="Name" )

# Dropping the irrelevant columns i.e. Team and Weight
# Those values were dropped since axis was set equal to 1 and
# the changes were made in the original data frame since inplace was True.
data_df.drop(["Team", "Weight"], axis = 1, inplace = True)

# Display the resulting dataframe
data_df.head(50)
```

Out[2]:

Name	Number	Position	Age	Height	College	Salary
Avery Bradley	0.0	PG	25.0	6-2	Texas	7730337.0
Jae Crowder	99.0	SF	25.0	6-6	Marquette	6796117.0
John Holland	30.0	SG	27.0	6-5	Boston University	NaN
R.J. Hunter	28.0	SG	22.0	6-5	Georgia State	1148640.0
Jonas Jerebko	8.0	PF	29.0	6-10	NaN	5000000.0
Amir Johnson	90.0	PF	29.0	6-9	NaN	12000000.0
Jordan Mickey	55.0	PF	21.0	6-8	LSU	1170960.0
Kelly Olynyk	41.0	C	25.0	7-0	Gonzaga	2165160.0
Terry Rozier	12.0	PG	22.0	6-2	Louisville	1824360.0
Marcus Smart	36.0	PG	22.0	6-4	Oklahoma State	3431040.0
Jared Sullinger	7.0	C	24.0	6-9	Ohio State	2569260.0
Isaiah Thomas	4.0	PG	27.0	5-9	Washington	6912869.0
Evan Turner	11.0	SG	27.0	6-7	Ohio State	3425510.0
James Young	13.0	SG	20.0	6-6	Kentucky	1749840.0
Tyler Zeller	44.0	C	26.0	7-0	North Carolina	2616975.0
Bojan Bogdanovic	44.0	SG	27.0	6-8	NaN	3425510.0
Markel Brown	22.0	SG	24.0	6-3	Oklahoma State	845059.0
Wayne Ellington	21.0	SG	28.0	6-4	North Carolina	1500000.0
Rondae Hollis-Jefferson	24.0	SG	21.0	6-7	Arizona	1335480.0
Jarrett Jack	2.0	PG	32.0	6-3	Georgia Tech	6300000.0
Sergey Karasev	10.0	SG	22.0	6-7	NaN	1599840.0
Sean Kilpatrick	6.0	SG	26.0	6-4	Cincinnati	134215.0
Shane Larkin	0.0	PG	23.0	5-11	Miami (FL)	1500000.0
Brook Lopez	11.0	C	28.0	7-0	Stanford	19689000.0
Chris McCullough	1.0	PF	21.0	6-11	Syracuse	1140240.0
Willie Reed	33.0	PF	26.0	6-10	Saint Louis	947276.0
Thomas Robinson	41.0	PF	25.0	6-10	Kansas	981348.0
Henry Sims	14.0	C	26.0	6-10	Georgetown	947276.0
Donald Sloan	15.0	PG	28.0	6-3	Texas A&M	947276.0
Thaddeus Young	30.0	PF	27.0	6-8	Georgia Tech	11235955.0
Arron Afflalo	4.0	SG	30.0	6-5	UCLA	8000000.0

Name	Number	Position	Age	Height	College	Salary
Lou Amundson	17.0	PF	33.0	6-9	UNLV	1635476.0
Thanasis Antetokounmpo	43.0	SF	23.0	6-7	NaN	30888.0
Carmelo Anthony	7.0	SF	32.0	6-8	Syracuse	22875000.0
Jose Calderon	3.0	PG	34.0	6-3	NaN	7402812.0
Cleanthony Early	11.0	SF	25.0	6-8	Wichita State	845059.0
Langston Galloway	2.0	SG	24.0	6-2	Saint Joseph's	845059.0
Jerian Grant	13.0	PG	23.0	6-4	Notre Dame	1572360.0
Robin Lopez	8.0	C	28.0	7-0	Stanford	12650000.0
Kyle O'Quinn	9.0	PF	26.0	6-10	Norfolk State	3750000.0
Kristaps Porzingis	6.0	PF	20.0	7-3	NaN	4131720.0
Kevin Seraphin	1.0	C	26.0	6-10	NaN	2814000.0
Lance Thomas	42.0	SF	28.0	6-8	Duke	1636842.0
Sasha Vujacic	18.0	SG	32.0	6-7	NaN	947276.0
Derrick Williams	23.0	PF	25.0	6-8	Arizona	4000000.0
Tony Wroten	5.0	SG	23.0	6-6	Washington	167406.0
Elton Brand	42.0	PF	37.0	6-9	Duke	NaN
Isaiah Canaan	0.0	PG	25.0	6-0	Murray State	947276.0
Robert Covington	33.0	SF	25.0	6-9	Tennessee State	1000000.0
Joel Embiid	21.0	C	22.0	7-0	Kansas	4626960.0

In [3]:
```
# Example 2: Syntax Errors
# We can also remove any syntax error than we find in our records.
# Remove white spaces: Extra white spaces at the beginning or the end of a strin
# Pad strings: Strings can be padded with spaces or other characters to a certai
# Let's remove use the str.strip function on the respective column name to strip
#

# Creating the DataFrame to be used
df1 = {
    'County':['     Nairobi     ', ' Machakos       ', ' Kisumu', 'Nakuru
   'Score':[73, 83, 75, 84, 61]}

df1 = pd.DataFrame(df1,columns=['County','Score'])

# Displaying the resulting dataframe
df1['County'][0]
```

Out[3]:
```
'     Nairobi     '
```

In [4]:
```
# Example 2: Syntax Errors
# We can use the str.strip function on the respective column name to strip the l
#

df1['County'] = df1['County'].str.strip()

# Displaying the resulting dataframe
df1['County'][0]
```

Out[4]:
```
'Nairobi'
```

In [5]:
```
# Example 3: Syntax Errors
# We can also use str.replace() function on the respective column to strip all t
#

df1['County'] = df1['County'].str.replace(" ","")

# Displaying the resulting dataframe
df1
```

Out[5]:

	County	Score
0	Nairobi	73
1	Machakos	83
2	Kisumu	75
3	Nakuru	84
4	Naroko	61

In [6]:
```
# Example 4:
# Fix typos: Strings can be entered in many different ways, and no wonder they c
#

# If you want to replace certain words - "Naroko" with "Narok"
df1['County'] = df1['County'].str.replace('Naroko', 'Narok')

df1
```

Out[6]:

	County	Score
0	Nairobi	73
1	Machakos	83
2	Kisumu	75
3	Nakuru	84
4	Narok	61

In [7]:
```
# Example 5: Outliers
# Values that are significantly different from all other observations are called
# Any data value that lies more than (1.5 * IQR) away from the Q1 and Q3 quartil
# For example, When one student averages over 90% while the rest of the class is
# Outliers are innocent until proven guilty. With that being said,
# they should not be removed unless there is a good reason for that.
```

```
# They are worth investigating before removing.
#

# Making the dataframe from csv file
outliers_df = pd.read_csv("http://bit.ly/BostonHousingDataset")

boston_df = pd.DataFrame(outliers_df)

outliers_df.describe()
```

Out[7]:

	crim	zn	indus	chas	nox	rm	a
count	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000(
mean	3.613524	11.363636	11.136779	0.069170	0.554695	6.284634	68.5749
std	8.601545	23.322453	6.860353	0.253994	0.115878	0.702617	28.1488
min	0.006320	0.000000	0.460000	0.000000	0.385000	3.561000	2.900(
25%	0.082045	0.000000	5.190000	0.000000	0.449000	5.885500	45.025(
50%	0.256510	0.000000	9.690000	0.000000	0.538000	6.208500	77.500(
75%	3.677083	12.500000	18.100000	0.000000	0.624000	6.623500	94.075(
max	88.976200	100.000000	27.740000	1.000000	0.871000	8.780000	100.000(

In [8]:

```
# Example 5: Outliers - Box plot
# We can also discover outliers with visualization tools
# A box plot is a method for graphically depicting groups of numerical data thro
# Box plots may also have lines extending vertically from the boxes (whiskers)
# indicating variability outside the upper and lower quartiles.
# In this example, we will plot outliers as individual points.
#

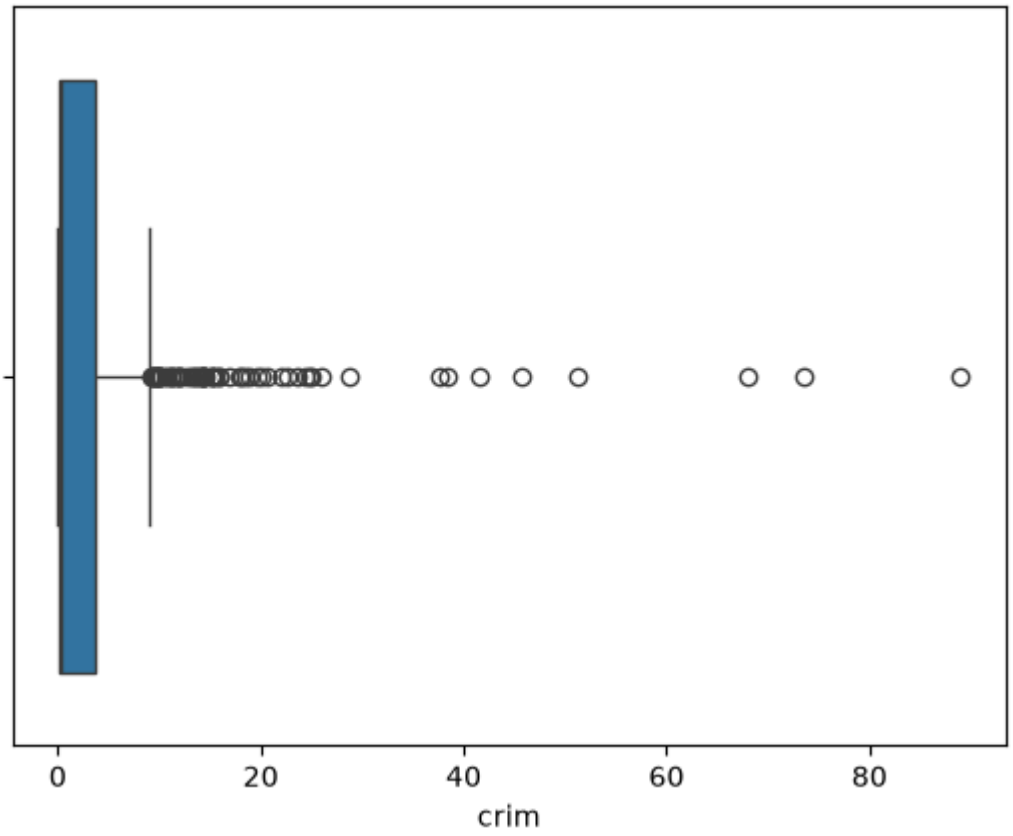
# We will use seaborn library which is a python visualisation library
import seaborn as sns

# We will analyse uni-variate outlier i.e. we will use dis column only to check
#
sns.boxplot(x = outliers_df['crim'])

# The plot shows three points between 10 to 12,
# these are outliers as there are not included in the box of other observation
# i.e no where near the quartiles.
```

Out[8]:

```
<Axes: xlabel='crim'>
```



In [9]:
```
# Example 6: Outliers - Box plot (Multiple variables)
# We can also find outliers of multiple variables by creating a Box plot as show

# Reading data from url as dataframe
gapminder = pd.read_csv("http://bit.ly/FiveYearData")
# gapminder.describe()

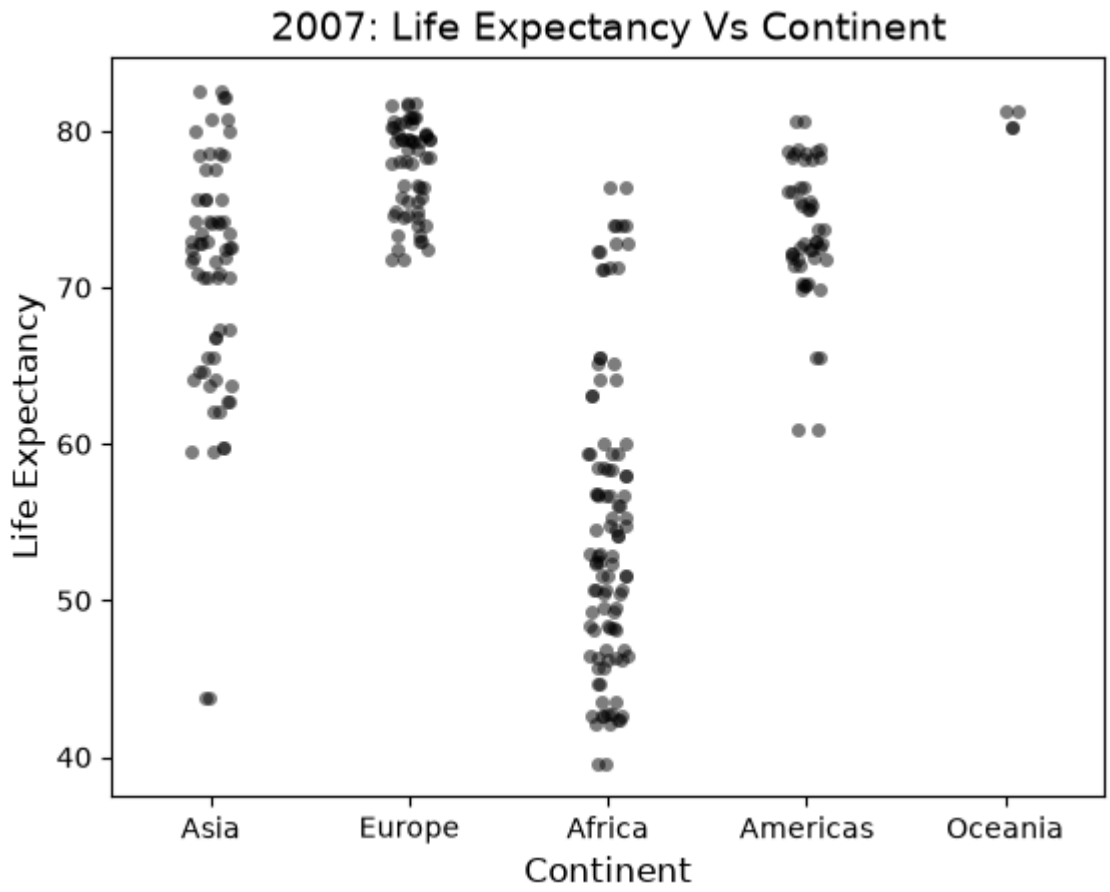
# # Filtering the data such that we will keep data from all countries for the ye
gapminder_2007 = gapminder[gapminder['year']==2007]
# gapminder_2007.head()

# We then specify from the multiple options to customize the boxplot with Seabor
# We choose color palette scheme for the boxplot with Seaborn.
# Here, we chose colorblind friendly palette "colorblind".
# Other color palette options available in Seaborn include deep, muted, bright,
# Uncomment the line below after running the previous lines
bplot=sns.stripplot(y='lifeExp', x='continent',  data=gapminder_2007, jitter=Tru

# Boxplot alone is extremely useful in getting the summary of data within and be
# However, often, it is a good practice to overlay the actual data points on the
# We use jitter = True option to spread the data points horizontally and also
# make boxplot with data points using stripplot.
# Uncomment the line below after running the previous lines
bplot=sns.stripplot(y='lifeExp', x='continent',  data=gapminder_2007, jitter=Tru

# We then name and size our graph and axis
# Uncomment the lines below after running the previous lines
bplot.axes.set_title("2007: Life Expectancy Vs Continent", fontsize=14)
bplot.set_xlabel("Continent", fontsize=12)
bplot.set_ylabel("Life Expectancy", fontsize=12)
```

Out[9]:
```
Text(0, 0.5, 'Life Expectancy')
```



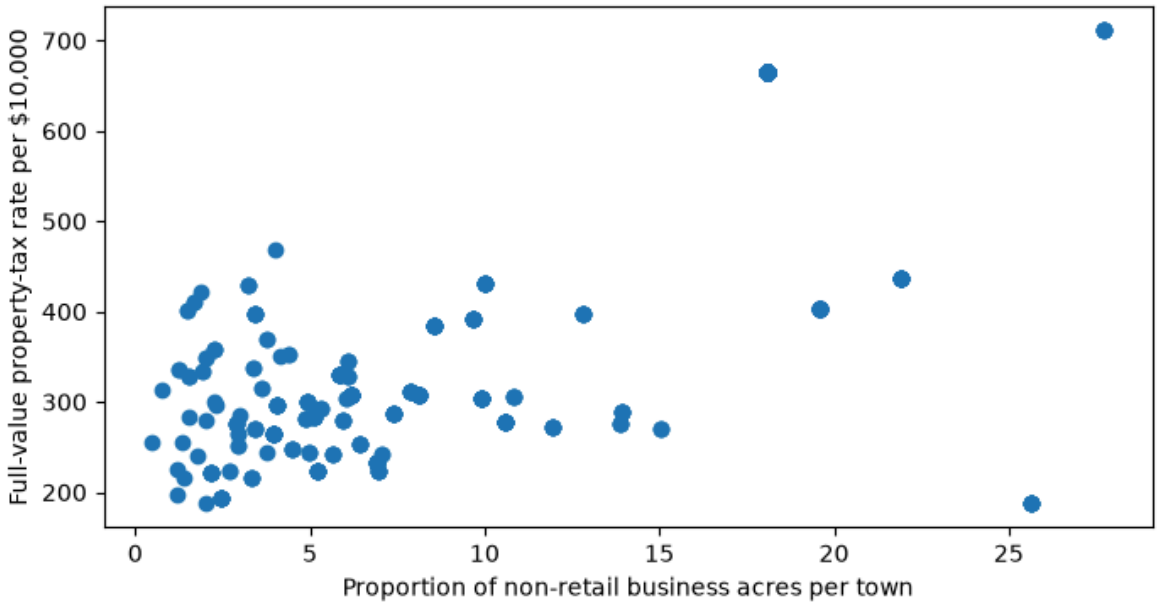
In [10]:

```
# Example 7: Outliers - Scatter plot
# The scatter plot is a collection of points that shows values for two variables
# We can try and draw scatter plot for two variables from our housing dataset.
#

# we will use matplotlib which is a plotting library
import matplotlib.pyplot as plt

# creating our scatterplot below
# Uncomment the lines below after running the previous line
fig, ax = plt.subplots(figsize=(8,4))
ax.scatter(boston_df['indus'], boston_df['tax'])
ax.set_xlabel('Proportion of non-retail business acres per town')
ax.set_ylabel('Full-value property-tax rate per $10,000')
plt.show()

# Looking at the plot below, we can see most of data points are lying bottom lef
# but there are points which are far from the population like top right corner.
```



In [11]:
```
# Example 8: Outliers - Mathematical Function (Z-Score-)
# The Z-score is the signed number of standard deviations by which the value of
# or data point is above the mean value of what is being observed or measured.
# The intuition behind Z-score is to describe any data point by finding their
# relationship with the Standard Deviation and Mean of the group of data points.
# Z-score is finding the distribution of data where mean is 0
# and standard deviation is 1 i.e. normal distribution.
# You must be wondering that, how does this help in identifying the outliers?
# Well, while calculating the Z-score we re-scale and center the data
# and look for data points which are too far from zero.
# These data points which are way too far from zero will be treated as the outli
# In most of the cases a threshold of 3 or -3 is used i.e
# if the Z-score value is greater than or less than 3 or -3 respectively,
# that data point will be identified as an outlier.

# We will use Z-score function defined in scipy library to detect the outliers.
# Scipy is another python library
#
from scipy import stats

z = np.abs(stats.zscore(boston_df))
z
```

Out[11]:
```
array([[0.41978194, 0.28482986, 1.2879095 , ..., 0.44105193, 1.0755623 ,
        0.15968566],
       [0.41733926, 0.48772236, 0.59338101, ..., 0.44105193, 0.49243937,
        0.10152429],
       [0.41734159, 0.48772236, 0.59338101, ..., 0.39642699, 1.2087274 ,
        1.32424667],
       ...,
       [0.41344658, 0.48772236, 0.11573841, ..., 0.44105193, 0.98304761,
        0.14880191],
       [0.40776407, 0.48772236, 0.11573841, ..., 0.4032249 , 0.86530163,
        0.0579893 ],
       [0.41500016, 0.48772236, 0.11573841, ..., 0.44105193, 0.66905833,
        1.15724782]], shape=(506, 14))
```

In [12]:
```
# Example 9: Outliers - Mathematical Function (Z-Score-)
# Looking the code and the output above, it is difficult to say which data point
# Let's try and define a threshold to identify an outlier.
```

```
#
threshold = 3
print(np.where(z > 3))

# The first array below contains the list of row numbers and second array respec
# which mean z[55][1] have a Z-score higher than 3.
```

```
(array([ 55,  56,  57, 102, 141, 142, 152, 154, 155, 160, 162, 163, 199,
       200, 201, 202, 203, 204, 208, 209, 210, 211, 212, 216, 218, 219,
       220, 221, 222, 225, 234, 236, 256, 257, 262, 269, 273, 274, 276,
       277, 282, 283, 283, 284, 347, 351, 352, 353, 353, 354, 355, 356,
       357, 358, 363, 364, 364, 365, 367, 369, 370, 372, 373, 374, 374,
       380, 398, 404, 405, 406, 410, 410, 411, 412, 412, 414, 414, 415,
       416, 418, 418, 419, 423, 424, 425, 426, 427, 427, 429, 431, 436,
       437, 438, 445, 450, 454, 455, 456, 457, 466]), array([ 1,  1,  1, 11, 12,
3,  3,  3,  3,  3,  3,  3,  1,  1,  1,  1,  1,
        1,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,  3,  5,  3,  3,  1,  5,
        5,  3,  3,  3,  3,  3,  3,  1,  3,  1,  1,  7,  7,  1,  7,  7,  7,
        3,  3,  3,  3,  3,  5,  5,  5,  3,  3,  3, 12,  5, 12,  0,  0,  0,
        0,  5,  0, 11, 11, 11, 12,  0, 12, 11, 11,  0, 11, 11, 11, 11, 11,
       11,  0, 11, 11, 11, 11, 11, 11, 11, 11, 11, 11, 11, 11, 11]))
```

In [13]:
```
# Example 10: Outliers - Mathematical Function (Z-Score-)
#

print(z[55][1])

# The data point – 55th record on column ZN is an outlier
```

```
3.375038763517294
```

In [14]:
```
# Example 11: Outliers - IQR
# We can also identify outliers using the interquartile range (IQR).
# This is the first quartile subtracted from the third quartile or simply put,
# the interquartile range (IQR is the range covered by the middle 50% of the dat
# The first and third quartile can be clearly seen on a box plot on the data.
# It is a measure of the dispersion similar to standard deviation or variance,
# but is much more robust against outliers.
# First, let's calculate IQR for each column.
#

Q1 = boston_df.quantile(0.25)
Q3 = boston_df.quantile(0.75)
IQR = Q3 - Q1

print(IQR)
```

```
crim         3.595038
zn          12.500000
indus       12.910000
chas         0.000000
nox          0.175000
rm           0.738000
age         49.050000
dis          3.088250
rad         20.000000
tax        387.000000
ptratio      2.800000
b           20.847500
lstat       10.005000
medv         7.975000
dtype: float64
```

In [15]:
```
# Example 12: Outliers - IQR
# As we now have the IQR scores, it's time to get hold on outliers.
# The below code will give an output with some true and false values.
# The data point where we have False that means these values are valid
# whereas True indicates presence of an outlier.
#
print((boston_df < (Q1 - 1.5 * IQR)) |(boston_df > (Q3 + 1.5 * IQR)))

# Now that we know how to detect the outliers,
# it is important to know whether if they need to be removed or corrected,
# or even left the way they are.
```

```
      crim     zn  indus   chas    nox     rm    age    dis    rad    tax  \
0    False  False  False  False  False  False  False  False  False  False
1    False  False  False  False  False  False  False  False  False  False
2    False  False  False  False  False  False  False  False  False  False
3    False  False  False  False  False  False  False  False  False  False
4    False  False  False  False  False  False  False  False  False  False
..     ...    ...    ...    ...    ...    ...    ...    ...    ...    ...
501  False  False  False  False  False  False  False  False  False  False
502  False  False  False  False  False  False  False  False  False  False
503  False  False  False  False  False  False  False  False  False  False
504  False  False  False  False  False  False  False  False  False  False
505  False  False  False  False  False  False  False  False  False  False

     ptratio      b  lstat   medv
0      False  False  False  False
1      False  False  False  False
2      False  False  False  False
3      False  False  False  False
4      False  False  False  False
..       ...    ...    ...    ...
501    False  False  False  False
502    False  False  False  False
503    False  False  False  False
504    False  False  False  False
505    False  False  False  False

[506 rows x 14 columns]
```

In [16]:
```
# Example 13: Removing the outliers (Z-Score)
# During data analysis when we detect the outlier one of most difficult decision
# could be how we should deal with the outlier.
# Should we remove them or correct them?
```

```
# We will learn more about how to handle outliers later in our readings.
# For now, let's see how we can remove them.
# One of the methods that we can use to remove our outliers is through the Z-sco
# as shown below
#

# we want to use a copy of boston_df
boston_zf = boston_df

# removing the outliers
boston_zf = boston_zf [(z < 3).all(axis=1)]

# boston_df.describe()
boston_zf.describe()
```

Out[16]:

	crim	zn	indus	chas	nox	rm	age	
count	415.000000	415.000000	415.000000	415.0	415.000000	415.000000	415.000000	4
mean	2.452461	9.602410	10.829687	0.0	0.547168	6.267554	67.384337	
std	4.897241	19.677404	6.798613	0.0	0.110428	0.611602	28.072043	
min	0.006320	0.000000	0.460000	0.0	0.385000	4.368000	2.900000	
25%	0.081005	0.000000	5.130000	0.0	0.449000	5.886000	43.550000	
50%	0.211610	0.000000	8.560000	0.0	0.524000	6.193000	73.900000	
75%	2.152115	12.500000	18.100000	0.0	0.609000	6.559500	93.700000	
max	28.655800	80.000000	27.740000	0.0	0.871000	8.337000	100.000000	

In [17]:
```
# Example 14: Removing the outliers (Z-Score)
# From above we have removed around 80+ rows from the dataset
# i.e. outliers have been removed.
#

# examining the shape of our old dataset
print(boston_df.shape)

# examining the shape of our cleaned dataset
print(boston_zf.shape)
```

```
(506, 14)
(415, 14)
```

In [18]:
```
outliers_df = pd.read_csv("http://bit.ly/BostonHousingDataset")
boston_df = pd.DataFrame(outliers_df)
Q1 = boston_df.quantile(0.25)
Q3 = boston_df.quantile(0.75)
IQR = Q3 - Q1
boston_df_iqr = boston_df[~((boston_df < (Q1 - 1.5 * IQR)) |
 (boston_df > (Q3 + 1.5 * IQR))).any(axis=1)]
print(boston_df_iqr.shape)
```

```
(268, 14)
```

1.1 Challenges

In [19]:
```
# Challenge 1
# We have been given the following dataset to work with
# url = http://bit.ly/MSOriginOfRefugees
# and we will be answering the following question;
# Which territories of origin had the highest no. of refugees from the year 2005
# Provide the relevant data for your analysis below.
# NB: We are only expected to clean our data. Once finished compare answers with

#OUR CODE GOES HERE
```

In [20]:
```
# Challenge 2
# Use the lstrip() function of .str method to check the variables with the leadi
# of the given NBA dataset below.
# Later, strip those leading and trailing spaces in the identified variables.
# url = http://bit.ly/MSDS-NBADataset

#OUR CODE GOES HERE
```

In [21]:
```
# Challenge 3
# Remove the text [edit] at the end of the university towns use sing the replace
# Use the following dataset; url = http://bit.ly/MSUniversityTown

#OUR CODE GOES HERE
```

In [22]:
```
# Challenge 4
# Using a Box - Plot determine and remove the outliers in the following dataset
# url = http://bit.ly/MSClimateDataset

#OUR CODE GOES HERE
```

In [23]:
```
# Challenge 5
# Find the outliers for variables in the following dataset for the year 1997 and
# Deal with the outliers respectively

#OUR CODE GOES HERE
```

In [24]:
```
# Challenge 6
# Attached is a set of products in which a sales and marketing team are trying t
# which products we should continue to sell, and which products to remove from o
# The file contains BOTH historical sales data AND active inventory,
# which can be discerned with the column titled "File Type".
# Find and deal with outliers in this dataset;
# url = http://bit.ly/MSProductSales

#OUR CODE GOES HERE
```

In [25]:
```
# Challenge 7
# Brazilian politicians are entitled to refunds if they spend any of their money
# on an activity that is enabling them to "better serve the people".
# Those expenses are public data, however, there is little monitoring/data analy
# A quick look at it shows a deputy with over 800 flights in one year.
# Another deputy has over 140.000R$ expenses on mailing (old fashion mail) in on
# There are a lot of very suspicious data regarding the deputies expending behav
# Can you help spot outliers and companies charging unusual amounts of money for
# Dataset url = http://bit.ly/MSBrazilPoliticians

import pandas as pd
import urllib.request
```

```python
url = "http://bit.ly/MSBrazilPoliticians"

req = urllib.request.Request(
    url,
    headers={"User-Agent": "Mozilla/5.0"}
)

with urllib.request.urlopen(req) as response:
    df = pd.read_csv(response)

print("Dataset preview:")
print(df.head())

print("\nColumn names:")
print(df.columns.tolist())
```

```
Dataset preview:
                      deputy_name deputy_state political_party  \
0  PROFESSORA DORINHA SEABRA REZENDE           TO             DEM
1               JOSÉ CARLOS ALELUIA           BA             DEM
2               JOSÉ CARLOS ALELUIA           BA             DEM
3               JOSÉ CARLOS ALELUIA           BA             DEM
4               JOSÉ CARLOS ALELUIA           BA             DEM

  refund_description                              company_name    company_id  \
0      FLIGHT TICKET                           Cia Aérea - TAM  2.012862e+12
1         FUEL COSTS  CASCOL COMBUSTIVEIS PARA VEICULOS LTDA  3.065970e+11
2         FUEL COSTS  CASCOL COMBUSTIVEIS PARA VEICULOS LTDA  3.065970e+11
3         FUEL COSTS            MEGA POSTO BERIMBAU LTDA - ME  1.855960e+12
4         FUEL COSTS            MEGA POSTO BERIMBAU LTDA - ME  1.855960e+12

           refund_date  refund_value    party_pg   party_en party_tse  \
0  2016-07-29T00:00:00        937.63  Democratas  Democrats        25
1  2016-06-21T00:00:00        150.00  Democratas  Democrats        25
2  2016-06-01T00:00:00        100.00  Democratas  Democrats        25
3  2016-04-19T00:00:00        168.07  Democratas  Democrats        25
4  2016-08-13T00:00:00        239.57  Democratas  Democrats        25

  party_regdate party_nmembers      party_ideology1       party_ideology2  \
0      11-09-86        1095712  Economic liberalism  Liberal conservatism
1      11-09-86        1095712  Economic liberalism  Liberal conservatism
2      11-09-86        1095712  Economic liberalism  Liberal conservatism
3      11-09-86        1095712  Economic liberalism  Liberal conservatism
4      11-09-86        1095712  Economic liberalism  Liberal conservatism

       party_ideology3 party_ideology4 party_position
0  Christian democracy             NaN      Rightwing
1  Christian democracy             NaN      Rightwing
2  Christian democracy             NaN      Rightwing
3  Christian democracy             NaN      Rightwing
4  Christian democracy             NaN      Rightwing

Column names:
['deputy_name', 'deputy_state', 'political_party', 'refund_description', 'company
_name', 'company_id', 'refund_date', 'refund_value', 'party_pg', 'party_en', 'par
ty_tse', 'party_regdate', 'party_nmembers', 'party_ideology1', 'party_ideology2',
'party_ideology3', 'party_ideology4', 'party_position']
```

```
C:\Users\Neeza\AppData\Local\Temp\ipykernel_16328\3173091766.py:22: DtypeWarning:
Columns (0: party_tse, 1: party_nmembers, 2: party_ideology2, 3: party_ideology3,
4: party_ideology4) have mixed types. Specify dtype option on import or set low_m
emory=False.
  df = pd.read_csv(response)
```

1.2 Accuracy

1.2 Challenge

In [26]:

```python
# Challenge 1: In-record & cross-datasets errors
# These errors result from having two or more values in the same row or across
# datasets that contradict with each other.
# For example, if we have a dataset about the cost of living in cities.
# The total column must be equivalent to the sum of rent, transport, and food.
# Similarly, a child can't be married. An employee's salary can't be less than t
# Using the given dataset below, determine and fix errors where the total popula
# total male population and total female population
# dataset url = http://bit.ly/MSPopulationDistribution
# Hint: Review Pandas Basics

#OUR CODE GOES HERE
import pandas as pd
import numpy as np
import urllib.request

url = "http://bit.ly/MSPopulationDistribution"

req = urllib.request.Request(
    url,
    headers={"User-Agent": "Mozilla/5.0"}
)

with urllib.request.urlopen(req) as response:
    df = pd.read_csv(response)

print("Original Data Preview")
print(df.head())

total_col = "Total_Population"
male_col = "Total_Male_Population"
female_col = "Total_Female_Population"

mismatch_mask = df[total_col] != (df[male_col] + df[female_col])
print("\nDetected discrepancies")
print(mismatch_mask.sum())

df.loc[mismatch_mask, total_col] = df[male_col] + df[female_col]

errors_remaining = (df[total_col] != (df[male_col] + df[female_col])).sum()
logical_breaches = (
    (df[male_col] > df[total_col]) | (df[female_col] > df[total_col])
).sum()


print("\nData Cleaning Quality Audit")
print("Remaining mathematical mismatches")
```

```
print(errors_remaining)
print("Remaining logical subset breaches")
print(logical_breaches)

print("\nCleaned Data Preview")
print(df.head())
```

Original Data Preview

	Urban_Center	District	Status	Core-Urban_Male_Population	Core-Urban_Female_Population	Total_Core-Urban_Population	Peri-Urban_Male_Population	Peri-Urban_Female_Population	Total_Peri-Urban_Population	Rural_Male_Population	Rural_Female_Population	Total_Rural_Population	Total_Male_Population	Total_Female_Population	Total_Population	OBJECTID
0	NAIROBI	Nairobi East/West/North/Westlands	City	1602104	1531414	3133518	NaN	NaN	NaN	NaN	NaN	NaN	1602104	1531414	3133518	0
1	MOMBASA	Mombasa/Kilindini	City	473433	441668	915101	12775.0	10255.0	23030.0	NaN	NaN	NaN	486208	451923	938131	1
2	KISUMU	Kisumu East/West/Nyando	City	131062	128196	259258	62816.0	66237.0	129053.0	10356.0	11261.0	21617.0	204234	205694	409928	2
3	NAKURU	Nakuru/Nakuru North	Municipality	145038	141373	286411	10843.0	10736.0	21579.0	NaN	NaN	NaN	155881	152109	307990	3
4	ELDORET	Eldoret East/West/Wareng	Municipality	127808	124253	252061	18788.0	18531.0	37319.0	NaN	NaN	NaN	146596	142784	289380	4

Detected discrepancies
0

Data Cleaning Quality Audit
Remaining mathematical mismatches
0
Remaining logical subset breaches
0

Cleaned Data Preview

	Urban_Center	District	Status
0	NAIROBI	Nairobi East/West/North/Westlands	City
1	MOMBASA	Mombasa/Kilindini	City
2	KISUMU	Kisumu East/West/Nyando	City
3	NAKURU	Nakuru/Nakuru North	Municipality
4	ELDORET	Eldoret East/West/Wareng	Municipality

```
   Core-Urban_Male_Population  Core-Urban_Female_Population  \
0                     1602104                       1531414
1                      473433                        441668
2                      131062                        128196
3                      145038                        141373
4                      127808                        124253

   Total_Core-Urban_Population  Peri-Urban_Male_Population  \
0                      3133518                         NaN
1                       915101                     12775.0
2                       259258                     62816.0
3                       286411                     10843.0
4                       252061                     18788.0

   Peri-Urban_Female_Population  Total_Peri-Urban_Population  \
0                           NaN                          NaN
1                       10255.0                      23030.0
2                       66237.0                     129053.0
3                       10736.0                      21579.0
4                       18531.0                      37319.0

   Rural_Male_Population  Rural_Female_Population  Total_Rural_Population  \
0                    NaN                      NaN                     NaN
1                    NaN                      NaN                     NaN
2                10356.0                  11261.0                 21617.0
3                    NaN                      NaN                     NaN
4                    NaN                      NaN                     NaN

   Total_Male_Population  Total_Female_Population  Total_Population  OBJECTID
0                1602104                  1531414           3133518         0
1                 486208                   451923            938131         1
2                 204234                   205694            409928         2
3                 155881                   152109            307990         3
4                 146596                   142784            289380         4
```

In [27]:
```
print(df.columns.tolist())
```

```
['Urban_Center', 'District', 'Status', 'Core-Urban_Male_Population', 'Core-Urban_
Female_Population', 'Total_Core-Urban_Population', 'Peri-Urban_Male_Population',
'Peri-Urban_Female_Population', 'Total_Peri-Urban_Population', 'Rural_Male_Popula
tion', 'Rural_Female_Population', 'Total_Rural_Population', 'Total_Male_Populatio
n', 'Total_Female_Population', 'Total_Population', 'OBJECTID']
```

1.3 Completeness

In [28]:
```
# Example 1: Checking/ Counting Missing Values

# # Creating a dataframe to work with
df1 = {
    'Subject':['Humanities', 'Physical Education', 'Home Science', 'Mathematics'
               'Chemistry','Arts'],
   'Score':[62, 47, np.nan, 74, np.nan, 77, 85]}

df1 = pd.DataFrame(df1,columns=['Subject','Score'])

print(df1)

# Checking if there is any missing value in dataframe as a whole
# Uncomment the following line after running the previous lines
```

```
# df1.isnull()

# Checking if there is any missing value across each column
# Uncomment the following line after running the previous lines
# df1.isnull().any()

# Checking how many missing values there are across each column
# Uncomment the following line after running the previous lines
# df1.isnull().sum()

# Or we can do a quick check to see if we have any missing values at all
# Uncomment the following line after running the previous lines
# print(df1.isnull().values.any())

# We might also want to get a total count of missing values
# Uncomment the following line after running the previous lines
# print(df1.isnull().sum().sum())
```

```
              Subject  Score
0          Humanities   62.0
1  Physical Education   47.0
2        Home Science    NaN
3         Mathematics   74.0
4             Physics    NaN
5           Chemistry   77.0
6                Arts   85.0
```

In [29]:
```
# Example 2: Dropping Missing Values
# If there are only a few null values and you know that deleting values
# will not cause adverse effects on your result,
# remove them from your DataFrame and store that in a new DataFrame

# Dropping all
# Uncomment the following 2 lines after running the previous lines
clean_df = df1.dropna()
print(clean_df)

# Verifying that you no longer have any null values by running
# Uncomment the following line after running the previous lines
# print(clean_df.isnull().sum())
```

```
              Subject  Score
0          Humanities   62.0
1  Physical Education   47.0
3         Mathematics   74.0
5           Chemistry   77.0
6                Arts   85.0
```

In [30]:
```
# Example 2: Dropping instances/records/rows with NA missing values
#

# Dropping all rows with any NA values
all_rows_any_na = df1.dropna()
print(all_rows_any_na)

# Dropping all rows that have all NA values
# Uncomment the 2 lines below after running the previous lines
# all_rows_all_na = df1.dropna(how="all")
# print(all_rows_all_na)

# We can also put a limitation on how many non-null values need to be in a row
```

```
# we can retain the data that has at least 2 non-null values as shown below
# Uncomment the 2 lines below after running the previous lines
# df1_thresh = df1.dropna(thresh=2)
# print(df1_thresh)
```

```
             Subject  Score
0          Humanities   62.0
1  Physical Education   47.0
3         Mathematics   74.0
5           Chemistry   77.0
6                Arts   85.0
```

In [31]:
```
subjects= {
  'Subject':['Humanities', 'Physical Education', 'Home Science', 'Mathematics',
             'Chemistry','Arts'],
  'Score':[62, 47, np.nan, 74, np.nan, 77, 85]}
subjects_df = pd.DataFrame(subjects,columns=['Subject','Score'])
subjects_df['Score'] = subjects_df['Score'].fillna((subjects_df['Score'].mean())
print(subjects_df)
```

```
             Subject  Score
0          Humanities   62.0
1  Physical Education   47.0
2       Home Science   69.0
3         Mathematics   74.0
4             Physics   69.0
5           Chemistry   77.0
6                Arts   85.0
```

In [32]:
```
# Example 4: Missing Values
# Imputing the attribute median for all missing values
#

# Creating a dataframe to work with
# subjects_df_median = {
#     'Subject':['Humanities', 'Physical Education', 'Home Science', 'Mathematic
#                'Chemistry','Arts'],
#    'Score':[62, 47, np.nan, 74, np.nan, 77, 85]}
# subjects_df_median = pd.DataFrame(subjects_df_median,columns=['Subject','Score

# printing our dataframe
# print(subjects_df_median)

# imputing the median
# subjects_df_median['Score'] = subjects_df_median['Score'].fillna((subjects_df_

# printing out our updated dataframe
# print(subjects_df_median)
```

In [33]:
```
# Example 5: Missing Values
# Imputing the attribute mode for all missing values

# Creating a dataframe to work with
subjects_df_mode = {
    'Subject':['Humanities', 'Physical Education', 'Home Science', 'Mathematics'
               'Chemistry','Arts'],
   'Score':[62, 47, np.nan, 74, np.nan, 77, 85]}
subjects_df_mode = pd.DataFrame(subjects_df_mode,columns=['Subject','Score'])

# printing our dataframe
```

```
print(subjects_df_mode)

# imputing the mode
subjects_df_mode['Score'] = subjects_df_mode['Score'].fillna((subjects_df_mode['

# printing out our updated dataframe
print(subjects_df_mode)
```

```
             Subject  Score
0          Humanities   62.0
1  Physical Education   47.0
2        Home Science    NaN
3         Mathematics   74.0
4             Physics    NaN
5           Chemistry   77.0
6                Arts   85.0
             Subject  Score
0          Humanities   62.0
1  Physical Education   47.0
2        Home Science   74.0
3         Mathematics   74.0
4             Physics   85.0
5           Chemistry   77.0
6                Arts   85.0
```

In [34]:
```
# Example 6: Missing Values
# Or maybe we just want to fill in missing values with a single value as shown b
#

df1 = {
    'Subject':['Humanities', 'Physical Education', 'Home Science', 'Mathematics'
               'Chemistry','Arts'],
   'Score':[62, 47, np.nan, 74, np.nan, 77, 85]}

df1 = pd.DataFrame(df1,columns=['Subject','Score'])

# Replace missing values with a number
df1['Score'].fillna(60, inplace=True)

print(df1)
```

```
             Subject  Score
0          Humanities   62.0
1  Physical Education   47.0
2        Home Science    NaN
3         Mathematics   74.0
4             Physics    NaN
5           Chemistry   77.0
6                Arts   85.0
```

```
C:\Users\Neeza\AppData\Local\Temp\ipykernel_16328\1280361557.py:13: ChainedAssign
mentError: A value is being set on a copy of a DataFrame or Series through chaine
d assignment using an inplace method.
Such inplace method never works to update the original DataFrame or Series, becau
se the intermediate object on which we are setting values always behaves as a cop
y (due to Copy-on-Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.meth
od({col: value}, inplace=True)' instead, to perform the operation inplace on the
original object, or try to avoid an inplace operation using 'df[col] = df[col].me
thod(value)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/
pandas-docs/stable/user_guide/copy_on_write.html
  df1['Score'].fillna(60, inplace=True)
```

1.3 Challenges

In [35]:
```
# Challenge 1
# Clean the following dataset that contains missing values
# url = http://bit.ly/DSRussianInvestigation
#
#OUR CODE GOES HERE
```

In [36]:
```
# Challenge 2
# Examine and clean the avengers dataset
# http://bit.ly/MSAvengers
#
#OUR CODE GOES HERE
```

In [37]:
```
# Challenge 3
# Clean the dataset where 16 multiple choice ability items were taken from the S
# web based personality assessment project. The data from 1525 subjects are incl
# set for scoring multiple choice inventories and doing basic item statistics.
# dataset url = http://bit.ly/MSIQQuestions
#
#OUR CODE GOES HERE
```

In [38]:
```
# Challenge 4
# The dataset below contains data from an experiment conducted by the Department
# at Kenyon College at a site near the campus in Gambier, Ohio. A student and fa
# planted 1000 white pine (Pinus strobes) seedlings at the Brown Family Environm
# These seedlings were planted in two grids, distinguished by 10- and 15-foot sp
# Several variables were measured and recorded for each seedling over time (in 1
# Handle the missing values of this dataset; dataset url = http://bit.ly/MSPines
#
# Features Description
# Row   Row number in pine plantation
# Col   Column number in pine plantation
# Hgt90 Tree height at time of planting (cm)
# Hgt96 Tree height in September 1996 (cm)
# Diam96        Tree trunk diameter in September 1996 (cm)
# Grow96        Leader growth during 1996 (cm)
# Hgt97 Tree height in September 1997 (cm)
# Diam97        Tree trunk diameter in September 1997 (cm)
# Spread97      Widest lateral spread in September 1997 (cm)
# Needles97     Needle length in September 1997 (mm)
# Deer95        Type of deer damage in September 1995: 0 = none, 1 = browsed
```

```
# Deer97        Type of deer damage in September 1997: 0 = none, 1 = browsed
# Cover95       Thorny cover in September 1995: 0 = none; 1 = some; 2 = moderate
# Fert  Indicator for fertilizer: 0 = no, 1 = yes
# Spacing       Distance (in feet) between trees (10 or 15)
#
#OUR CODE GOES HERE
```

In [39]:

```
# Challenge 5
# Some geologists were interested in the water chemistry of rivers in Coastal Ne
# They took water samples at three different locations in four rivers.
# The sampling sites were chosen to investigate how the composition of the water
# as it flows from the source to the mouth of each river.
# The sampling sites were labeled as upstream, midstream, and downstream.
# This dataset contains the concentrations (parts per million) of a variety of e
# The dataset RiverIron contains the information for iron (FE) alone, along with
# Handle the missing values found in this dataset; dataset url = http://bit.ly/M

# Features Description
# River One of four rivers: Grasse, Oswegatchie, Raquette, or St. Regis
# Site  Location: 1=UpStream, 2=MidStream, 3=Downstream
# Al    Aluminum
# Ba    Barium
# Br    Bromine
# Ca    Calcium
# Ce    Cerium
# Cu    Copper
# Dy    Dysprosium
# Er    Erbim
# Fe    Iron
# Gd    Gadolinium
# Ho    Holmum
# K     Potassium
# La    Lathanum
# Li    Lithium
# Mg    Magnesium
# Mn    Manganese
# Nd    Neodymium
# Pr    Proseyodymium
# Rb    Rubidium
# Si    Silicon
# Sr    Strontium
# Y     Yttrium
# Yb    Ytterbium
# Zn    Zinc
# Zr    Zirconium
#
#OUR CODE GOES HERE
import pandas as pd
import numpy as np

url = "http://bit.ly/MSRiverSamples"
df = pd.read_csv(url)

print("--- Dataset Info ---")
print(df.info())
print("\n--- First 5 Rows ---")
print(df.head())

print("\n--- Missing Values Count per Column ---")
print(df.isnull().sum())
```

```
if 'FE' in df.columns:
    df['log_FE'] = np.log(df['FE'] + 1e-5)
    print("\nLog transformation applied to Iron ('FE'). New column 'log_FE' crea
else:
    print("\nWarning: 'FE' column not found in the loaded dataset.")

print(df.head())
```

```
--- Dataset Info ---
<class 'pandas.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 28 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   rownames  12 non-null     int64
 1   River     12 non-null     str
 2   Site      12 non-null     int64
 3   Al        12 non-null     int64
 4   Ba        12 non-null     float64
 5   Br        12 non-null     int64
 6   Ca        12 non-null     int64
 7   Ce        12 non-null     float64
 8   Cu        12 non-null     float64
 9   Dy        12 non-null     float64
 10  Er        12 non-null     float64
 11  Fe        12 non-null     int64
 12  Gd        12 non-null     float64
 13  Ho        11 non-null     float64
 14  K         12 non-null     int64
 15  La        12 non-null     float64
 16  Li        12 non-null     float64
 17  Mg        12 non-null     int64
 18  Mn        12 non-null     float64
 19  Nd        12 non-null     float64
 20  Pr        12 non-null     float64
 21  Rb        12 non-null     float64
 22  Si        12 non-null     int64
 23  Sr        12 non-null     float64
 24  Y         12 non-null     float64
 25  Yb        12 non-null     float64
 26  Zn        12 non-null     float64
 27  Zr        12 non-null     float64
dtypes: float64(18), int64(9), str(1)
memory usage: 2.8 KB
None

--- First 5 Rows ---
   rownames        River  Site   Al     Ba  Br     Ca    Ce   Cu    Dy  ...  \
0         1       Grasse     1  231  13.20   8   3602  0.92  0.3  0.14  ...
1         2       Grasse     2  166  13.82   6   6432  0.93  0.5  0.16  ...
2         3       Grasse     3   83  13.39  10  10707  0.52  1.3  0.10  ...
3         4  Oswegatchie     1  373  11.66   8   2262  1.76  0.5  0.21  ...
4         5  Oswegatchie     2   71  13.77  29   7862  0.35  1.0  0.07  ...

      Mn    Nd    Pr    Rb    Si     Sr     Y    Yb    Zn    Zr
0   2.18  0.76  0.18  0.58  3482  19.35  1.15  0.10  10.0  0.10
1   2.34  0.93  0.22  0.94  3230  24.80  1.24  0.09   4.1  0.08
2   1.40  0.56  0.13  1.05  3343  41.88  0.72  0.07   2.1  0.09
3  16.69  1.37  0.32  0.80  2937  11.21  1.71  0.15  10.0  0.11
4   1.33  0.40  0.10  1.10  2636  34.35  0.57  0.04   6.3  0.05

[5 rows x 28 columns]

--- Missing Values Count per Column ---
rownames    0
River       0
Site        0
Al          0
```

```
Ba          0
Br          0
Ca          0
Ce          0
Cu          0
Dy          0
Er          0
Fe          0
Gd          0
Ho          1
K           0
La          0
Li          0
Mg          0
Mn          0
Nd          0
Pr          0
Rb          0
Si          0
Sr          0
Y           0
Yb          0
Zn          0
Zr          0
dtype: int64

Warning: 'FE' column not found in the loaded dataset.
   rownames        River  Site   Al     Ba  Br     Ca    Ce   Cu    Dy  ...  \
0         1       Grasse     1  231  13.20   8   3602  0.92  0.3  0.14  ...
1         2       Grasse     2  166  13.82   6   6432  0.93  0.5  0.16  ...
2         3       Grasse     3   83  13.39  10  10707  0.52  1.3  0.10  ...
3         4  Oswegatchie     1  373  11.66   8   2262  1.76  0.5  0.21  ...
4         5  Oswegatchie     2   71  13.77  29   7862  0.35  1.0  0.07  ...

      Mn    Nd    Pr    Rb    Si     Sr     Y    Yb    Zn    Zr
0   2.18  0.76  0.18  0.58  3482  19.35  1.15  0.10  10.0  0.10
1   2.34  0.93  0.22  0.94  3230  24.80  1.24  0.09   4.1  0.08
2   1.40  0.56  0.13  1.05  3343  41.88  0.72  0.07   2.1  0.09
3  16.69  1.37  0.32  0.80  2937  11.21  1.71  0.15  10.0  0.11
4   1.33  0.40  0.10  1.10  2636  34.35  0.57  0.04   6.3  0.05

[5 rows x 28 columns]
```

1.4 Consistency

In [40]:

```
# Example 1: Duplicates
# Duplicates are data points that are repeated in your dataset.
# These should be simply removed.
#

raw_data = {'first_name': ['Jamila', 'Jamila', 'Jane','Joma', 'Ulia', 'Dorcas'],
        'last_name': ['Mohammed', 'Mohammed', 'Milner','Tech', 'Shevchenko', 'Wa
        'age': [52, 52, 43, 77, 25, 19],
        'pre_assessment': [5, 5, 5, 43, 5, 2],
        'post_assessment': [35, 35, 25, 57, 72, 71]}
df_duplicate = pd.DataFrame(raw_data, columns = ['first_name', 'last_name', 'age
print(df_duplicate)
```

```
# Identifying which observations are duplicates
df_non_duplicates = df_duplicate.drop_duplicates()

# Let's see these non-duplicate files
print(df_non_duplicates)
```

```
  first_name   last_name  age  pre_assessment  post_assessment
0     Jamila    Mohammed   52               5               35
1     Jamila    Mohammed   52               5               35
2       Jane      Milner   43               5               25
3       Joma        Tech   77              43               57
4       Ulia  Shevchenko   25               5               72
5     Dorcas     Wanjiru   19               2               71
  first_name   last_name  age  pre_assessment  post_assessment
0     Jamila    Mohammed   52               5               35
2       Jane      Milner   43               5               25
3       Joma        Tech   77              43               57
4       Ulia  Shevchenko   25               5               72
5     Dorcas     Wanjiru   19               2               71
```

1.4 Challenges

In [41]:

```
# Challenge  1
# Identify and handle the duplicted records found in the following dataset
# dataset url = http://bit.ly/MoviesDataset1
#
#OUR CODE GOES HERE
# Challenge 1
# Identify and handle the duplicated records found in the following dataset
# dataset url = http://bit.ly/MoviesDataset1

import pandas as pd

# Load the real dataset directly using your assignment link
movies_df = pd.read_csv("http://bit.ly/MoviesDataset1")

print("--- Original Movies DataFrame ---")
print(movies_df)

# 1. Identify which rows are duplicates
print("\n--- Identifying Duplicate Rows (True means it is a duplicate) ---")
print(movies_df.duplicated())

# 2. Handle duplicates by dropping them
clean_movies_df = movies_df.drop_duplicates()

print("\n--- Cleaned Movies DataFrame (Duplicates Removed) ---")
print(clean_movies_df)
```

```
--- Original Movies DataFrame ---
                                    movie_title  num_critic_for_reviews  \
0                                       Avatar?ÿ                   723
1     Pirates of the Caribbean: At World's End?ÿ                   302
2                                      Spectre?ÿ                   602
3                        The Dark Knight Rises?ÿ                   813
4                                    John Carter?ÿ                   462
5                                   Spider-Man 3?ÿ                   392
6                                        Tangled?ÿ                   324
7                         Avengers: Age of Ultron?ÿ                   635
8                         Avengers: Age of Ultron?ÿ                   635
9       Harry Potter and the Half-Blood Prince?ÿ                   375
10          Batman v Superman: Dawn of Justice?ÿ                   673
11                             Superman Returns?ÿ                   434
12                            Quantum of Solace?ÿ                   403
13  Pirates of the Caribbean: Dead Man's Chest?ÿ                   313

    duration DIRECTOR_facebook_likes  actor_3_facebook_likes  \
0      178.0                      10                     855
1        NaN                     563                    1000
2      148.0                      20                     161
3        NaN                   22000                   23000
4      132.0                   "475"                     530
5      156.0                      23                    4000
6        NaN                      15                     284
7      141.0                      10                   19000
8      141.0                      10                   19000
9      153.0                     282                   10000
10     183.0                     NaN                    2000
11     169.0                     NaN                     903
12     106.0                     395                     393
13     151.0                     563                    1000

    ACTOR_1_facebook_likes      gross  num_voted_users  \
0                     1000  760505847         886204.0
1                    40000  309404152         471220.0
2                    11000  200074175         275868.0
3                    27000  448130642        1144337.0
4                      640   73058679         212204.0
5                    24000  336530303         383056.0
6                      799  200807262         294810.0
7                    26000  458991599         462669.0
8                    26000  458991599         462669.0
9                    25000  301956980         321795.0
10                   15000  330249062              NaN
11                   18000  200069408         240396.0
12                     451  168368427         330784.0
13                   40000  423032628         522040.0

    Cast_Total_facebook_likes  facenumber_in_poster  num_user_for_reviews  \
0                      4834.0                   NaN                  3054
1                     48350.0                   NaN                  1238
2                     11700.0                   1.0                   994
3                    106759.0                   NaN                  2701
4                      1873.0                   1.0                   738
5                     46055.0                   NaN                  1902
6                         NaN                   1.0                   387
7                     92000.0                   4.0                  1117
8                     92000.0                   4.0                  1117
9                     58753.0                   3.0                   973
```

```
10                   24450.0                   NaN                  3018
11                       NaN                   2.0                  2367
12                    2023.0                   1.0                  1243
13                   48486.0                   2.0                  1832

       budget  title_year  ACTOR_2_facebook_likes  imdb_score  title_year.1
0   237000000        2009                   936.0         7.9        2009.0
1   300000000        2007                  5000.0         7.1           NaN
2   245000000        2015                   393.0         6.8        2015.0
3   250000000        2012                 23000.0         8.5           NaN
4   263700000        2012                   632.0         6.6           NaN
5   258000000        2007                 11000.0         6.2        2007.0
6   260000000        2010                   553.0         7.8           NaN
7   250000000        2015                 21000.0         7.5           NaN
8   250000000        2015                 21000.0         7.5        2015.0
9   250000000        2009                 11000.0         7.5           NaN
10  250000000        2016                     NaN         6.9        2016.0
11  209000000        2006                 10000.0         6.1           NaN
12  200000000        2008                   412.0         6.7        2008.0
13  225000000        2006                  5000.0         7.3        2008.0

--- Identifying Duplicate Rows (True means it is a duplicate) ---
0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9     False
10    False
11    False
12    False
13    False
dtype: bool

--- Cleaned Movies DataFrame (Duplicates Removed) ---
                                     movie_title  num_critic_for_reviews  \
0                                       Avatar?ÿ                   723
1     Pirates of the Caribbean: At World's End?ÿ                   302
2                                      Spectre?ÿ                   602
3                          The Dark Knight Rises?ÿ                   813
4                                    John Carter?ÿ                   462
5                                   Spider-Man 3?ÿ                   392
6                                        Tangled?ÿ                   324
7                         Avengers: Age of Ultron?ÿ                   635
8                         Avengers: Age of Ultron?ÿ                   635
9       Harry Potter and the Half-Blood Prince?ÿ                   375
10          Batman v Superman: Dawn of Justice?ÿ                   673
11                             Superman Returns?ÿ                   434
12                           Quantum of Solace?ÿ                   403
13  Pirates of the Caribbean: Dead Man's Chest?ÿ                   313

    duration DIRECTOR_facebook_likes  actor_3_facebook_likes  \
0      178.0                      10                    855
1        NaN                     563                   1000
2      148.0                      20                    161
3        NaN                   22000                  23000
```

```
4      132.0                  "475"                   530
5      156.0                    23                   4000
6        NaN                    15                    284
7      141.0                    10                  19000
8      141.0                    10                  19000
9      153.0                   282                  10000
10     183.0                   NaN                   2000
11     169.0                   NaN                    903
12     106.0                   395                    393
13     151.0                   563                   1000

    ACTOR_1_facebook_likes      gross  num_voted_users  \
0                     1000  760505847         886204.0
1                    40000  309404152         471220.0
2                    11000  200074175         275868.0
3                    27000  448130642        1144337.0
4                      640   73058679         212204.0
5                    24000  336530303         383056.0
6                      799  200807262         294810.0
7                    26000  458991599         462669.0
8                    26000  458991599         462669.0
9                    25000  301956980         321795.0
10                   15000  330249062              NaN
11                   18000  200069408         240396.0
12                     451  168368427         330784.0
13                   40000  423032628         522040.0

    Cast_Total_facebook_likes  facenumber_in_poster  num_user_for_reviews  \
0                      4834.0                   NaN                  3054
1                     48350.0                   NaN                  1238
2                     11700.0                   1.0                   994
3                    106759.0                   NaN                  2701
4                      1873.0                   1.0                   738
5                     46055.0                   NaN                  1902
6                         NaN                   1.0                   387
7                     92000.0                   4.0                  1117
8                     92000.0                   4.0                  1117
9                     58753.0                   3.0                   973
10                    24450.0                   NaN                  3018
11                        NaN                   2.0                  2367
12                     2023.0                   1.0                  1243
13                    48486.0                   2.0                  1832

       budget  title_year  ACTOR_2_facebook_likes  imdb_score  title_year.1
0   237000000        2009                   936.0         7.9        2009.0
1   300000000        2007                  5000.0         7.1           NaN
2   245000000        2015                   393.0         6.8        2015.0
3   250000000        2012                 23000.0         8.5           NaN
4   263700000        2012                   632.0         6.6           NaN
5   258000000        2007                 11000.0         6.2        2007.0
6   260000000        2010                   553.0         7.8           NaN
7   250000000        2015                 21000.0         7.5           NaN
8   250000000        2015                 21000.0         7.5        2015.0
9   250000000        2009                 11000.0         7.5           NaN
10  250000000        2016                     NaN         6.9        2016.0
11  209000000        2006                 10000.0         6.1           NaN
12  200000000        2008                   412.0         6.7        2008.0
13  225000000        2006                  5000.0         7.3        2008.0
```

In [42]:
```
# Clean the following dataset
# dataset url = http://bit.ly/EmployeesDataset
#
#OUR CODE GOES HERE
employees_df = pd.read_csv("http://bit.ly/EmployeesDataset")

employees_df.drop_duplicates(inplace=True)

if 'employee_id' in employees_df.columns:
    employees_df.drop_duplicates(subset=['employee_id'], keep='last', inplace=Tr

print("Employee records streamlined.")
```

```
Employee records streamlined.
```

1.5 Uniformity

In [43]:
```
# Example 1: Standardization - Renaming column names
# We can rename multiple  data frame column names in the following manner
#

# Create an example dataframe
#
officials_data = {'Moderator': ['Wanjiku', 'Muthoni', 'Kagure', 'Muema', 'Kariuk
        'Date': ['2015-02-11', '2013-02-22', '2015-02-11', '2014-02-11', '2011-0
        'Score': [14, 34, 41, 12, 13]}
officials_data = pd.DataFrame(officials_data)
print(officials_data)

# Renaming the column names

officials_data.columns = ['Senior Leader', 'Time', 'Score']
print(officials_data)
```

```
  Moderator        Date  Score
0   Wanjiku  2015-02-11     14
1   Muthoni  2013-02-22     34
2    Kagure  2015-02-11     41
3     Muema  2014-02-11     12
4   Kariuki  2011-02-11     13
  Senior Leader        Time  Score
0       Wanjiku  2015-02-11     14
1       Muthoni  2013-02-22     34
2        Kagure  2015-02-11     41
3         Muema  2014-02-11     12
4       Kariuki  2011-02-11     13
```

In [44]:
```
# Example 2: Standardization - Fixing messy column names
# Sometimes you might have column names which are uppercase, with spaces,
# and whitespace all around. How do we fix this?
# We use the .str method that we use on text data.
# Ideally, we chain a bunch of .str functions as shown below;
#

# we use strip(), lower() and replace() functions
#
officials_data.columns = officials_data.columns.str.strip().str.lower().str.repl
print(officials_data)
```

```
  senior_leader        time  score
0       Wanjiku  2015-02-11     14
1       Muthoni  2013-02-22     34
2        Kagure  2015-02-11     41
3         Muema  2014-02-11     12
4       Kariuki  2011-02-11     13
```

In [45]:
```
# Example 3: Date Type Conversion
# Make sure numbers are stored as numerical data types.
# A date should be stored as a date object, or a Unix timestamp (number of secon
# Making the respecive conversion is advised.
#

# Let's convert our strings to Datetime data type
# importing datatime module
from datetime import datetime

# Creating our data frame
data = {'date': ['2015-05-01 18:47:05.069722', '2015-05-01 18:47:05.119994',
                 '2015-05-02 18:47:05.178768', '2015-05-02 18:47:05.230071',
                 '2015-05-02 18:47:05.230071', '2015-05-02 18:47:05.280592',
                 '2015-05-03 18:47:05.332662', '2015-05-03 18:47:05.385109',
                 '2015-05-04 18:47:05.436523', '2015-05-04 18:47:05.486877'],
        'value': [1, 2, 4, 5, 6, 3, 2, 1, 15, 11]}
df = pd.DataFrame(data, columns = ['date', 'value'])

# printing our dataframe
print(df.info())

# Converting date column from string to datetime
df['date'] = pd.to_datetime(df['date'])

# printing our updated dataframe
print(df.info())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   date    10 non-null     str
 1   value   10 non-null     int64
dtypes: int64(1), str(1)
memory usage: 292.0 bytes
None
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   date    10 non-null     datetime64[us]
 1   value   10 non-null     int64
dtypes: datetime64[us](1), int64(1)
memory usage: 292.0 bytes
None
```

In [46]:
```
# Example 4: Data Type conversion
# Sometimes, especially when we're reading in a CSV with a bunch of numbers,
# some of the numbers will read in as strings instead of numeric values, or vice
# Let's fix the release year to be a string and not a number;
```

```
# In pandas, the datatype string appears as object
#

# First let's see how our datatypes appear while loading the dataset
data = pd.read_csv('http://bit.ly/MovieMetaData')
print(data.info())

# Now let's specify that our 'duration column needs to be an integer value'
# Uncomment the lines below after running the previous lines
# data = pd.read_csv('http://bit.ly/MovieMetaData', dtype={'title_year': str})
# print(data.dtypes)
```

```
<class 'pandas.DataFrame'>
RangeIndex: 5043 entries, 0 to 5042
Data columns (total 28 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   color                      5024 non-null   str
 1   director_name              4939 non-null   str
 2   num_critic_for_reviews     4993 non-null   float64
 3   duration                   5028 non-null   float64
 4   director_facebook_likes    4939 non-null   float64
 5   actor_3_facebook_likes     5020 non-null   float64
 6   actor_2_name               5030 non-null   str
 7   actor_1_facebook_likes     5036 non-null   float64
 8   gross                      4159 non-null   float64
 9   genres                     5043 non-null   str
 10  actor_1_name               5036 non-null   str
 11  movie_title                5043 non-null   str
 12  num_voted_users            5043 non-null   int64
 13  cast_total_facebook_likes  5043 non-null   int64
 14  actor_3_name               5020 non-null   str
 15  facenumber_in_poster       5030 non-null   float64
 16  plot_keywords              4890 non-null   str
 17  movie_imdb_link            5043 non-null   str
 18  num_user_for_reviews       5022 non-null   float64
 19  language                   5029 non-null   str
 20  country                    5038 non-null   str
 21  content_rating             4740 non-null   str
 22  budget                     4551 non-null   float64
 23  title_year                 4935 non-null   float64
 24  actor_2_facebook_likes     5030 non-null   float64
 25  imdb_score                 5043 non-null   float64
 26  aspect_ratio               4714 non-null   float64
 27  movie_facebook_likes       5043 non-null   int64
dtypes: float64(13), int64(3), str(12)
memory usage: 1.1 MB
None
```

1.5 Challenges

In [47]:
```
# Challenge 1
# Rename and fix the columns names of the following dataset
# url = http://bit.ly/MSPropertyDataset
# Columns = Street number, Street name, Owner Occupied, No. of Bedrooms
#
#OUR CODE GOES HERE
```

In [48]:
```
# Challenge 2
# Convert the following dataframe column Data from string to datetime format
#

schedule_df = pd.DataFrame({'Date':['23/9/2011', '11/4/2010', '10/12/2019'],
                'Event':['Music', 'Drama', 'Arts'],
                'Cost':[17000, 55000, 25000]})

#OUR CODE GOES HERE
```

In [49]:
```
# Challenge 3
# Work on converting the following data types from the given sales dataset
# url = http://bit.ly/SalesDataset
# - The Customer Number is a float64 but it should be an int64
# - The 2016 and 2017 columns are stored as objects, not numerical values such a
# - Percent Growth and Jan Units are also stored as objects not numerical values
# - Month, Day and Year columns that should be converted to datetime64
# - Active column should be a boolean
# We cannot work with this dataset until we clean up normalize the datatypes
# A good example of this would be if we were to perform the following operation
#

# Loading our sales dataset
sales_df = pd.read_csv("http://bit.ly/SalesDataset")

for col in ['2016', '2017']:
    sales_df[col] = sales_df[col].astype(str).str.replace('$', '', regex=False).
    sales_df[col] = pd.to_numeric(sales_df[col], errors='coerce').fillna(0.0)

if 'Percent Growth' in sales_df.columns:
    sales_df['Percent Growth'] = sales_df['Percent Growth'].astype(str).str.repl
    sales_df['Percent Growth'] = pd.to_numeric(sales_df['Percent Growth'], error

if 'Customer Number' in sales_df.columns:
    sales_df['Customer Number'] = sales_df['Customer Number'].fillna(0).astype(n

if 'Active' in sales_df.columns:
    sales_df['Active'] = sales_df['Active'].astype(str).str.lower().map({'yes':

print("Standardized Sales Dataset Summary Types:")
print(sales_df.dtypes)
```

```
Standardized Sales Dataset Summary Types:
Customer Number      int64
Customer Name          str
2016               float64
2017               float64
Percent Growth     float64
Jan Units              str
Month                int64
Day                  int64
Year                 int64
Active              object
dtype: object
```